

ЛАБОРАТОРНАЯ РАБОТА: ИГРА "СБОРЩИК"

Цель работы: Создать браузерную игру, применяя объектно-ориентированный подход. Вы научитесь инкапсулировать логику в классы, обрабатывать события клавиатуры и работать с общим состоянием игры через объект данных.

Описание игры: Игрок спавнится на поле, вокруг него создаётся множество разноцветных "монеток" для сбора, игрок должен собрать все за ограниченное время.

ШАГ 1: ПОДГОТОВКА СЦЕНЫ И GAMEDATA

На этом этапе мы создадим холст и подготовим объект, который будет хранить всю важную информацию об игре.

1. Создайте три файла: `index.html`, `style.css` и `script.js`.
2. В HTML добавьте тег `canvas` с идентификатором `gameCanvas`.
3. Используя и CSS, и javascript, настройте размеры `canvas` по ширине и высоте экрана с помощью свойств `document.documentElement.clientWidth` (и `clientHeight`)
4. В JS найдите canvas по id и получите контекст `ctx = canvas.getContext('2d')`.
5. Создайте объект `gameData` со свойствами: `width`, `height`, `score: 0`, `totalCoins: 20`, `timeLeft: 15`, `isGameOver: false`. Запишите `width` и `height`, используя те же свойства `window`.

ЗАЧЕМ НАМ GAMEDATA?

Вместо того чтобы наши игровые классы (например, Игрок) брали размеры экрана или текущий счет напрямую из разрозненных глобальных переменных, мы будем передавать им объект `gameData`. Это правило хорошего тона в программировании. Так классы становятся независимыми: вы можете скопировать класс Игрока в другой проект, и он будет работать там с любыми размерами поля.

ШАГ 2: СОЗДАНИЕ КЛАССА ИГРОКА (PLAYER)

Мы создадим "чертеж" героя. Объект сам будет знать свои координаты и как себя рисовать.

1. Создайте класс `Player`.
2. В `constructor(x, y, size, speed, color)` сохраните параметры в свойства объекта через `this`.
3. Добавьте метод `draw(ctx)` для отрисовки квадрата через `fillRect`.
4. Создайте экземпляр: `let player = new Player(385, 285, 30, 5, 'blue')`.
5. Вызовите `player.draw(ctx)`.

Как проверить результат:

В центре холста должен появиться синий квадрат.

ШАГ 3: ОБРАБОТКА КЛАВИАТУРЫ И ДВИЖЕНИЕ

Заставим игру обновлять кадры и научим класс Player реагировать на клавиши, используя `gameData` для проверки границ.

1. Создайте объект `keys = {}`. Добавьте слушатели `keydown` и `keyup` на `window` для записи состояния клавиш.
2. В классе Player добавьте метод `update(keys, gameData)`.
3. Внутри `update` меняйте `this.x` и `this.y` в зависимости от нажатых клавиш (WASD/Стрелки).
4. Добавьте проверки: координаты не должны выходить за пределы 0 и `gameData.width/height`.
5. Создайте функцию `gameLoop()`: очистка холста, `update`, `draw` и `requestAnimationFrame(gameLoop)`.

Как проверить результат:

Персонаж должен плавно перемещаться и упираться в края холста.

ШАГ 4: СОЗДАНИЕ КЛАССА МОНЕТКИ (COIN)

Создадим класс для объектов, которые нужно собирать на поле.

1. Создайте класс `Coin` с конструктором (`x`, `y`, `radius`, `color`) и методом `draw(ctx)` (используйте `arc`).
2. Создайте пустой массив `coins = []`.
3. Напишите функцию `generateCoins(count, gameData)`, которая в цикле создает объекты `Coin` со случайными координатами и цветом.
4. Вызовите генерацию перед запуском цикла и отрисуйте монетки в `gameLoop`.

Как проверить результат:

На поле должна появиться россыпь разноцветных кружков.

ШАГ 5: СТОЛКНОВЕНИЯ И УСЛОВИЕ ПОБЕДЫ

Научим игру удалять собранные монетки и определять момент победы.

1. В `gameLoop` добавьте остановку: если `gameData.isGameOver`, то `return`.
2. Создайте функцию `checkCollisions(gameData)`. Используйте `Math.hypot` для расчета расстояния между игроком и каждой монеткой.
3. Если дистанция меньше суммы радиусов: удаляйте монетку через `splice` и прибавляйте `gameData.score`.
4. Если `score === totalCoins`: установите `isGameOver = true` и выведите текст "ВЫ ПОБЕДИЛИ!" зеленым цветом.

Как проверить результат:

Соберите все монетки — игра должна замереть и поздравить вас.

ШАГ 6: ИНТЕРФЕЙС И ТАЙМЕР (УСЛОВИЕ ПОРАЖЕНИЯ)

Добавим ограничение по времени и отрисовку текста прямо на холсте.

1. Создайте функцию `drawUI(gameData, ctx)`. Используйте `ctx.fillText` для вывода счета и `gameData.timeLeft`.
2. Создайте `setInterval` на 1000мс. В нем уменьшайте `timeLeft` на 1.
3. Если `timeLeft <= 0`: остановите таймер, установите `isGameOver = true` и выведите красный текст "ВРЕМЯ ВЫШЛО!".

Как проверить результат:

Проверьте оба сценария — победу при сборе всех монет и поражение при истечении времени.

ШАГ 7: РЕФАКТОРИНГ

Переработаем код, чтобы он был удобен в чтении, исправлении и доработке.

1. Поищите и исправьте **Магические значения** в коде - это числа и другие данные, которые обозначены сами по себе, например, `player.x += 5` - непонятно откуда взялась эта 5, что она означает. Исправить можно, используя константы или переменные для таких значений: `const playerSpeed = 5; player.x += playerSpeed;`
2. Поищите и исправьте дублирующийся код: один и тот же набор кода, где меняются лишь значения или небольшие части, может повторяться несколько раз, например классы могут иметь одни и те же свойства, подразумевающие одни и те же методы для работы с ними, либо вообще имеющие одни и те же методы, в таком случае можно выделить родительский класс, либо попробовать использовать композицию (но в нашем случае это не самый удобный способ); так же могут иметься просто независимые повторяющиеся части кода, их нужно будет выделить в отдельные функции.
3. Улучшить инкапсуляцию: некоторые значения, связанные какой-то логикой, находятся отдельно в глобальном пространстве. Можно выделить их в отдельный класс или файл.
4. Исправить смешивание или разделение логики: нужно осмотреться нет ли какой-то логики, которая по смыслу и действиям принадлежит каким-то классам, но при этом выделена и смешана в какую-то внешнюю техническую функцию.

Примечание:

Легко, пытаясь исправить одни проблемы, создать другие, в том числе и сломать работу кода, внимательно перепроверяйте что вы делаете. Если не знаете что еще можно исправить, спросите преподавателя.